Mapa del código

CU 28 "Registrar recepción de bolsín": cómo está armado, cómo funciona el patrón State y dónde está cada mensaje del diagrama de secuencia.

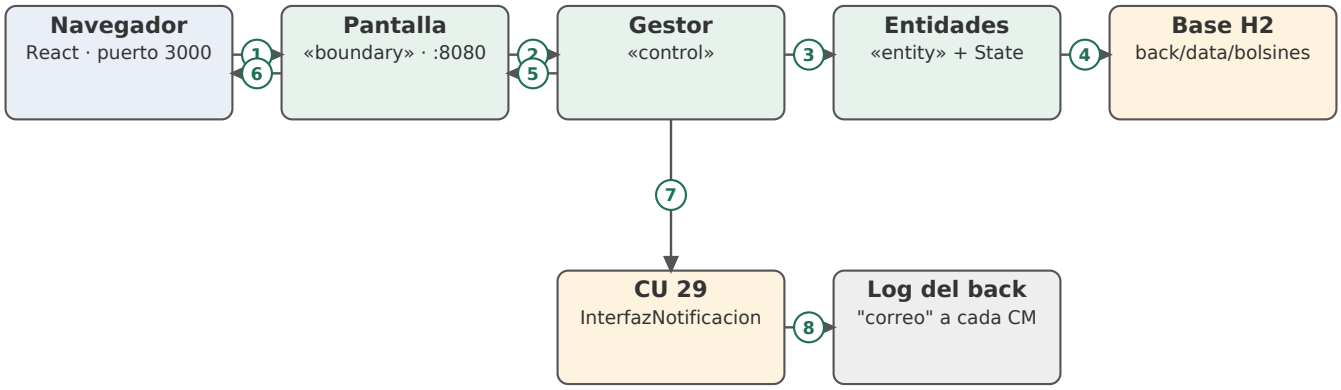
Cómo usar esta guía
Para entender el sistema: secciones 1 a 4, con un dibujo cada una.
Para seguir el diagrama de secuencia: sección 5. Cada mensaje tiene dónde se envía y dónde está implementado, en la forma **Clase**:línea.
Para "mostrame dónde está…": sección 8, todos los métodos de cada clase con su línea.
Para la defensa: sección 7 (lo que no es idéntico al diagrama y cómo explicarlo) y sección 10 (preguntas).
En IntelliJ: `Ctrl + N` abre una clase, `Ctrl + G` va a una línea y `Ctrl + B` sobre un método salta a su implementación.
Las líneas son las de la rama francisco-cu28 al 12/11. Si el código cambia, la guía se vuelve a generar.

Contenido

1. El sistema en una página
2. Dónde está cada cosa
3. Cómo viaja un mensaje
4. El patrón State
5. El diagrama de secuencia, mensaje por mensaje
6. Flujos alternativos
7. Lo que no es idéntico al diagrama
8. ¿Dónde está…? Índice por clase
9. Persistencia y datos de prueba
10. Preguntas típicas de la defensa

1. El sistema en una página

El EB (Encargado de Bolsines) elige un bolsín que llegó a su comisión médica, revisa los remitos y la documentación, elige una de las 4 opciones de recepción y confirma. El sistema cambia el estado del bolsín, de sus remitos y de cada documentación, y avisa a cada CM de origen (CU 29). Es **una sola aplicación en capas**: un back en Java con Spring Boot y un front en React que sólo dibuja lo que la Pantalla devuelve.

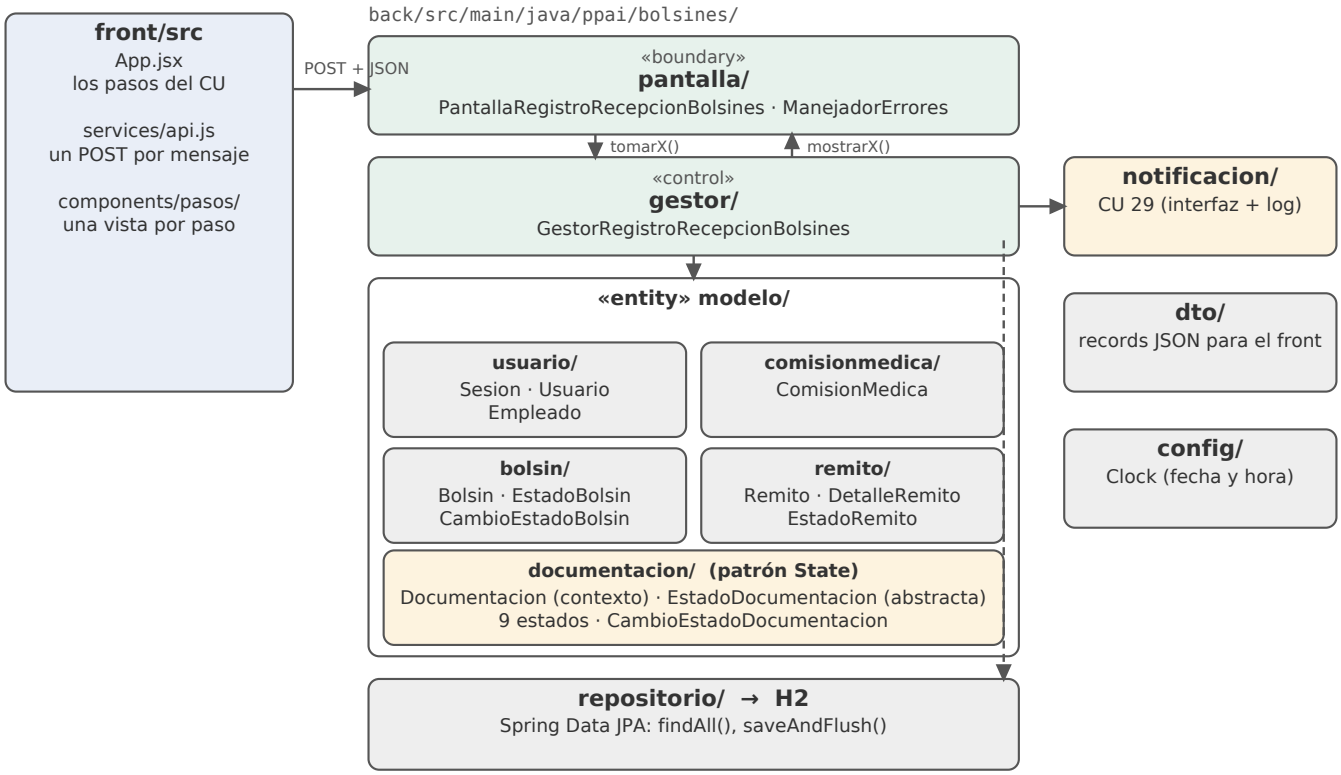


#	Qué pasa
1-2	Cada botón del EB es un POST a la Pantalla con el nombre del mensaje del diagrama, y la Pantalla se lo pasa al Gestor con el mismo nombre.
3-4	El Gestor coordina; las entidades responden sobre lo que conocen (Experto). JPA guarda los cambios en H2.
5-6	El Gestor le manda a la Pantalla los mostrarX() del diagrama; el endpoint lo devuelve como JSON y React lo dibuja.
7-8	llamarCasoUso29() prepara una notificación por CM de origen y el «include» las envía (se ven en el log).

Tres ideas para decir con estas palabras
El código sigue el diagrama de secuencia: cada mensaje es un método con el mismo nombre, parámetros, emisor y receptor. Las dos excepciones están explicadas en la sección 7.
El State está en Documentación: la documentación no decide con if; le delega a su estado actual.
Los alternativos son un agregado: métodos aparte, al final de cada archivo, sin clases nuevas.

2. Dónde está cada cosa

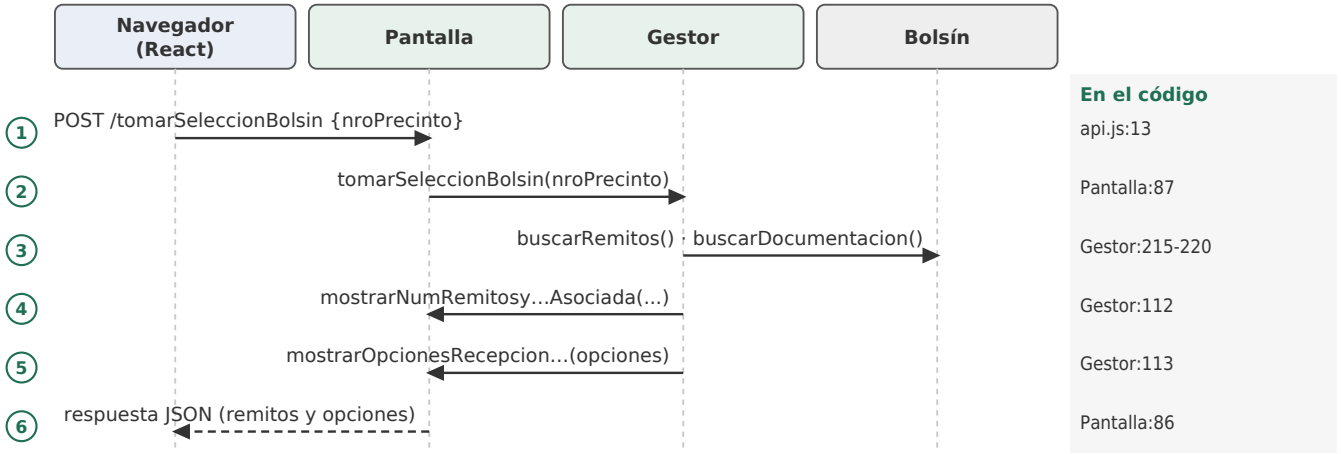
Repositorio `juangedda/PPAIDSIG6`: carpeta back (Java), `front` (React) y `docs` (diagramas, ERS y sesiones). En el back, cada paquete es una capa con el estereotipo del diagrama de clases:



Detalles que suelen preguntar
Pantalla y Gestor se conocen entre sí (tomarX en un sentido, mostrarX en el otro). Spring no puede crear dos objetos que se necesitan en el constructor, por eso el Gestor recibe la Pantalla con `@Lazy` (**Gestor**:48).
Los endpoints son `synchronized`**:** la Pantalla y el Gestor guardan lo que eligió el EB entre pedidos.
Cada archivo tiene dos partes: arriba los métodos del diagrama; al final, bajo el título "Flujos alternativos", los de A3, A4 y A5.

3. Cómo viaja un mensaje

Ejemplo: el EB elige un bolsín (paso 5 del CU). Las otras acciones funcionan igual.

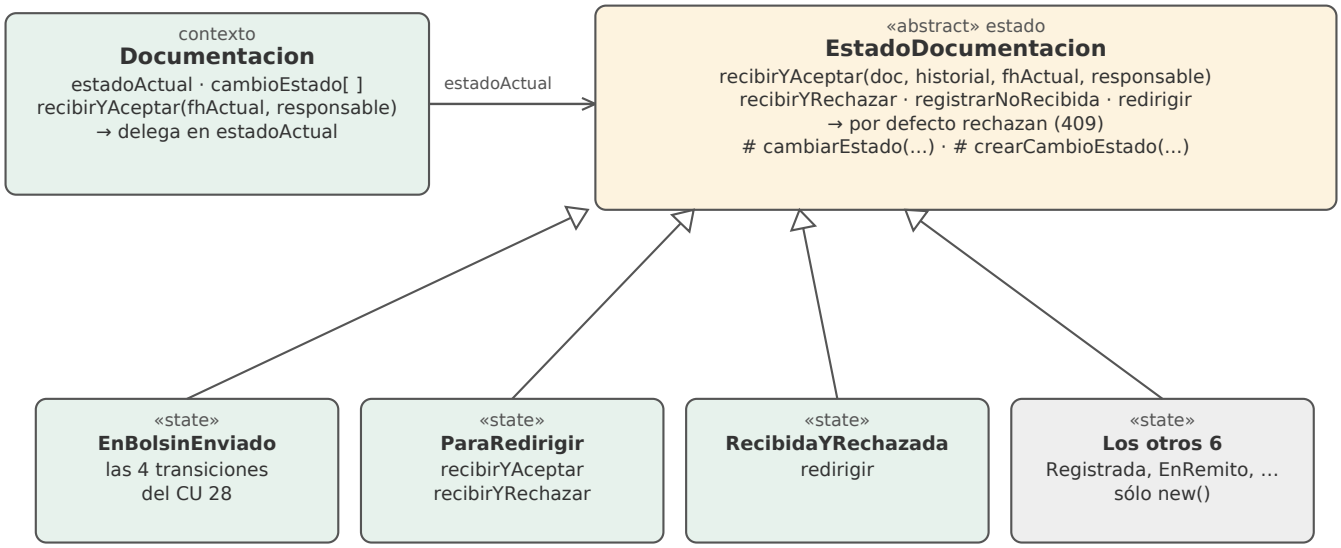


Acción del EB (paso del CU)	Front · api.js	Pantalla	Gestor
Registrar recepción (1-4)	opcRegistrarRecepcionBolsin · 10	**Pantalla**:52	**Gestor**:86
Elegir bolsín (5-7)	tomarSeleccionBolsin · 13	**Pantalla**:86	**Gestor**:100
Elegir opción (8-9)	tomarSeleccionOpcionRecepcion · 16	**Pantalla**:107	**Gestor**:121
Marcar documentación (A3-A5)	tomarSeleccionDocumentacion · 21	**Pantalla**:152	**Gestor**:332
Confirmar (10-12)	tomarConfirmacionRegistro · 24	**Pantalla**:124	**Gestor**:145

Si algo no está permitido, el código lanza una excepción y **ManejadorErrores** la devuelve como 400 (dato inválido) o 409 (no se puede en este momento, por ejemplo una transición que el estado no admite).

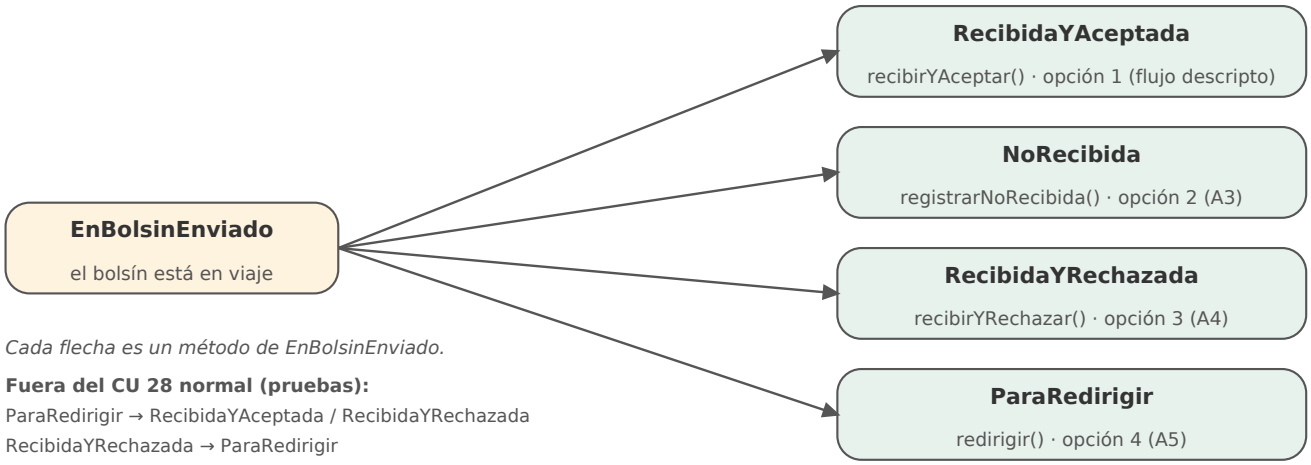
4. El patrón State

Problema: lo que se puede hacer con una documentación depende de su estado. Con if, cada operación preguntaría el estado, y agregar uno obligaría a tocarlas todas. **Solución:** una clase por estado; la documentación delega en su estado actual y cada estado concreto implementa sólo sus transiciones.



Cada estado concreto redefine sólo las transiciones que salen de él; el resto lo responde la clase abstracta (con un error).

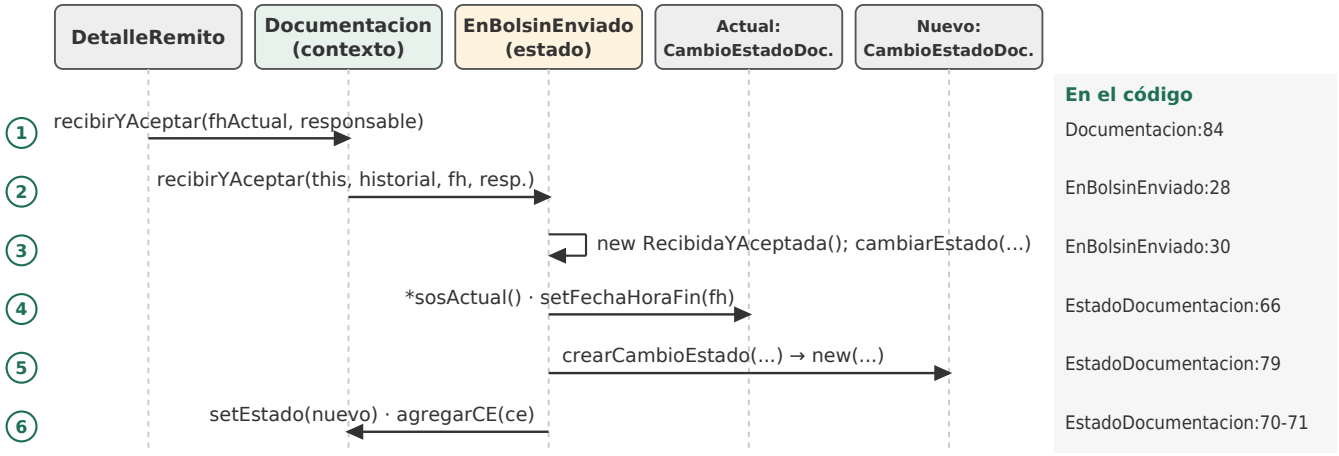
Las transiciones que usa el CU 28 (máquina de estados de Documentación):



Cada flecha es un método de EnBolsinEnviado.

Fuera del CU 28 normal (pruebas):
ParaRedirigir → RecibidaYAceptada / RecibidaYRechazada
RecibidaYRechazada → ParaRedirigir

Qué pasa cuando se recibe una documentación (paso 11 del CU)



CambioEstadoDoc. = CambioEstadoDocumentacion. cambiarEstado es el código común de la clase abstracta (abajo).

modelo/documentacion/EstadoDocumentacion.java · líneas 62 a 72

```
 62  protected void cambiarEstado(Documentacion doc, List<CambioEstadoDocumentacion> historial,
 63          LocalDateTime fhActual, Empleado responsable, EstadoDocumentacion nuevo) {
 64      for (CambioEstadoDocumentacion ce : historial) {
 65          if (ce.sosActual()) {
 66              ce.setFechaHoraFin(fhActual);
 67          }
 68      }
 69      CambioEstadoDocumentacion ce = crearCambioEstado(fhActual, nuevo, responsable);
 70      doc.setEstado(nuevo);
 71      doc.agregarCE(ce);
 72  }
```

Si la transición no es válida (por ejemplo, recibir una documentación ya recibida), el estado concreto no redefine el método y responde la clase abstracta (**EstadoDocumentacion**:31) con un error. El estado se guarda en la base por su nombre; **EstadoDocumentacionConverter** crea el objeto concreto al leer.

5. El diagrama de secuencia, mensaje por mensaje

El diagrama dividido en sus cuatro tramos, uno por cada acción del EB. En cada tramo: qué pasa, el código real del Gestor con el número de mensaje al costado (◄), y una fila por mensaje del diagrama. **Se envía en** es la línea donde está la llamada; **Implementado en**, la línea donde empieza el método que la recibe. Los * del diagrama son loops (un for).

A · Registrar recepción (pasos 1 a 4)

El EB abre la pantalla. El Gestor busca al usuario logueado y su CM, la muestra, y busca los bolsines enviados a esa CM. El número ◄ al costado del código es el mensaje del diagrama que se envía en esa línea.

gestor/GestorRegistroRecepcionBolsines.java · opcRegistrarRecepcionBolsin

```
86    public void opcRegistrarRecepcionBolsin() {
87        finCU(); // descarta un registro anterior que el EB canceló sin terminar (Obs. 1 del CU)
88        buscarUsuarioLogueado();                                                    ◄ 4
89        buscarCMUsuario();                                                          ◄ 8
90        pantalla.mostrarCM(cmUsuario.getNombre());                                  ◄ 10, 11
91        buscarBolsinesEnviados();                                                   ◄ 12
92        pantalla.mostrarBolsinesEnviadosParaSeleccion(nrosPrecinto, origenes);      ◄ 21
93    }
```

gestor/GestorRegistroRecepcionBolsines.java · buscarBolsinesEnviados

```
202   private void buscarBolsinesEnviados() {
203       bolsines = new ArrayList<>();
204       for (Bolsin bolsin : bolsinRepositorio.findAll()) {
205           if (bolsin.buscarBolsinesEnviados() && bolsin.esCmDestino(cmUsuario)) {   ◄ 13, 17
206               bolsines.add(bolsin);
207           }
208       }
209       nrosPrecinto = bolsines.stream().mapToInt(Bolsin::getNumPrecinto).toArray();  ◄ 18
210       origenes = bolsines.stream().map(Bolsin::getOrigen).toArray(String[]::new);   ◄ 19
211   }
```

#	Mensaje (emisor → receptor)	Se envía en	Implementado en	Nota
1	`opcRegistrarRecepcionBolsin()` EB → Pantalla	api.js:10	**Pantalla**:52	
2	`habilitarPantalla()` Pantalla → Pantalla	**Pantalla**:53	**Pantalla**:59	
3	`opcRegistrarRecepcionBolsin()` Pantalla → Gestor	**Pantalla**:54	**Gestor**:86	
4	`buscarUsuarioLogueado()` Gestor → Gestor	**Gestor**:88	**Gestor**:175	
5	`getUsuario(): Usuario` Gestor → Activa:Sesion	**Gestor**:178	**Sesion**:31	La sesión activa la trae el repositorio.
6	`getUsuario(): Usuario` Sesion → :Usuario	**Sesion**:32	**Usuario**:25	
7	`*esTuUsuario(u: Usuario): boolean` Gestor → :Empleado	**Gestor**:181	**Empleado**:35	
8	`buscarCMUsuario()` Gestor → Gestor	**Gestor**:89	**Gestor**:194	
9	`obtenerCM(): ComisionMedica` Gestor → Logueado:Empleado	**Gestor**:195	**Empleado**:43	
10	`getNombre(): String` Empleado → :ComisionMedica	**Gestor**:90	**ComisionMedica**:32	**Lo envía el Gestor** (sección 7).
11	`mostrarCM(nombre: String)` Gestor → Pantalla	**Gestor**:90	**Pantalla**:69	
12	`buscarBolsinesEnviados()` Gestor → Gestor	**Gestor**:91	**Gestor**:202	
13	`buscarBolsinesEnviados(): boolean` Gestor → :Bolsin (loop)	**Gestor**:205	**Bolsin**:69	
14	`*sosActual(): boolean` Bolsin → :CambioEstadoBolsin	**Bolsin**:71	**Cambio EstadoBolsin**:51	
15	`sosEnviado(): boolean` Bolsin → Actual:CambioEstadoBolsin	**Bolsin**:72	**Cambio EstadoBolsin**:56	
16	`sosEnviado(): boolean` CambioEstadoBolsin → :EstadoBolsin	**Cambio EstadoBolsin**:57	**EstadoBolsin**:23	
17	`esCmDestino(cm: ComisionMedica): boolean` Gestor → :Bolsin	**Gestor**:205	**Bolsin**:79	
18	`getNumPrecinto(): int` Gestor → :Bolsin	**Gestor**:209	**Bolsin**:84	
19	`getOrigen(): String` Gestor → :Bolsin	**Gestor**:210	**Bolsin**:92	
20	`getNombre(): String` Bolsin → :ComisionMedica	**Bolsin**:93	**ComisionMedica**:32	
21	`mostrarBolsinesEnviadosParaSeleccion (nroPrecinto: int[], origen: String[])` Gestor → Pantalla	**Gestor**:92	**Pantalla**:78	A1 y A2 filtran esta lista en el front.

B · Elegir un bolsín (pasos 5 a 7)

El EB elige un bolsín. El Gestor le pide al bolsín los números de sus remitos y los datos de la documentación, y muestra las 4 opciones de recepción.

`gestor/GestorRegistroRecepcionBolsines.java · tomarSeleccionBolsin`

```
100   public void tomarSeleccionBolsin(int nroPrecinto) {
101       if (bolsines == null) {
102           throw new IllegalStateException("Primero hay que elegir la opción de registrar la …
103       }
104       bolsinSeleccionado = bolsines.stream()
105               .filter(b -> b.getNumPrecinto() == nroPrecinto)
106               .findFirst()
107               .orElseThrow(() -> new IllegalArgumentException(
108                       "No se encontró un bolsín enviado con el número de precinto " + nroPre…
109       olvidarOpcion(); // la opción y lo marcado correspondían al bolsín anterior
110       buscarRemitos();                                                                    ◄ 24
111       buscarDocumentacion();                                                              ◄ 27
```

```
112       pantalla.mostrarNumRemitosyDocumentacionAsociada(nrosRemito, documentacion);                ◄ 34
113       pantalla.mostrarOpcionesRecepcionParaSeleccion(getOpcionesRecepcion());                     ◄ 35, 36
114   }
```

#	Mensaje (emisor → receptor)	Se envía en	Implementado en	Nota
22	`tomarSeleccionBolsin(nroPrecinto: int)` EB → Pantalla	api.js:13	**Pantalla**:86	
23	`tomarSeleccionBolsin(nroPrecinto: int)` Pantalla → Gestor	**Pantalla**:87	**Gestor**:100	
24	`buscarRemitos()` Gestor → Gestor	**Gestor**:110	**Gestor**:214	
25	`buscarRemitos(): int[]` Gestor → selec:Bolsin	**Gestor**:215	**Bolsin**:101	Devuelve los números.
26	`*getNumero(): int` Bolsin → :Remito	**Bolsin**:102	**Remito**:56	
27	`buscarDocumentacion()` Gestor → Gestor	**Gestor**:111	**Gestor**:219	
28	`buscarDocumentacion(): String[][]` Gestor → selec:Bolsin	**Gestor**:220	**Bolsin**:109	
29	`buscarDatosDocumentacion(): String[][]` Bolsin → :Remito (loop)	**Bolsin**:111	**Remito**:65	Filas [nroRemito, asunto, tipo].
30	`buscarDatosDocumentacion(): String[]` Remito → :DetalleRemito (loop)	**Remito**:68	**DetalleRemito**:40	
31	`getAsunto(): String` DetalleRemito → :Documentacion	**DetalleRemito**:41	**Documentacion**:66	
32	`getNombreTipo(): String` DetalleRemito → :Documentacion	**DetalleRemito**:41	**Documentacion**:71	
33	`getNombre(): String` Documentacion → :TipoDocumento	**Documentacion**:72	**TipoDocumento**:19	
34	`mostrarNumRemitosyDocumentacionAsociada(nrosRemito: int[], documentacion: String[][])` Gestor → Pantalla	**Gestor**:112	**Pantalla**:96	La Pantalla agrupa la documentación por remito.
35	`getOpcionesRecepcion(): String[]` Gestor → Gestor	**Gestor**:113	**Gestor**:224	
36	`mostrarOpcionesRecepcionParaSeleccion(opciones: String[])` Gestor → Pantalla	**Gestor**:113	**Pantalla**:101	

C · Elegir la opción 1 (pasos 8 y 9)

El EB elige "El contenido del bolsín es igual al registrado" y el Gestor pide la confirmación. Con las opciones 2, 3 y 4 va a los flujos alternativos (sección 6).

gestor/GestorRegistroRecepcionBolsines.java · tomarSeleccionOpcionRecepcion

```
121   public void tomarSeleccionOpcionRecepcion(String opcion) {
122       if (bolsinSeleccionado == null) {
123           throw new IllegalStateException("Primero hay que seleccionar un bolsín");
124       }
125       if (!List.of(getOpcionesRecepcion()).contains(opcion)) {
126           throw new IllegalArgumentException("Opción de recepción desconocida: " + opcion);
127       }
128       olvidarOpcion();
129       if (!opcion.equals(OPCIONES_RECEPCION[0])) { // opciones 2, 3 y 4: flujos alternativos…
130           solicitarSeleccionDocumentacion(opcion);
131           return;
132       }
133       opcRecepcion = opcion;
134       pantalla.solicitarConfirmacionRegistro();                                                   ◄ 39
135   }
```

#	Mensaje (emisor → receptor)	Se envía en	Implementado en	Nota
37	`tomarSeleccionOpcionRecepcion(opcion: String)` EB → Pantalla	api.js:16	**Pantalla**:107	

#	Mensaje (emisor → receptor)	Se envía en	Implementado en	Nota
38	`tomarSeleccionOpcionRecepcion(opcion : String)` Pantalla → Gestor	**Pantalla**:109	**Gestor**:121	Con la opción 2, 3 o 4 va a los alternativos (sección 6).
39	`solicitarConfirmacionRegistro()` Gestor → Pantalla	**Gestor**:134	**Pantalla**:114	

D · Confirmar: se actualiza todo (pasos 10 a 12)

El EB confirma. El Gestor busca los estados nuevos y la fecha, y le pide al bolsín que se actualice: el bolsín actualiza sus remitos, cada remito su documentación, y cada documentación le delega el cambio a su estado (State). Después se notifica (CU 29) y se muestra el éxito.

`gestor/GestorRegistroRecepcionBolsines.java · tomarConfirmacionRegistro`

```
145   public void tomarConfirmacionRegistro(boolean confirmacion) {
146       if (opcRecepcion == null) {
147           throw new IllegalStateException("Primero hay que elegir una opción de recepción");
148       }
149       if (!opcRecepcion.equals(OPCIONES_RECEPCION[0]) && nrosDocumentacionMarcada == null) {
150           throw new IllegalStateException("Primero hay que seleccionar la documentación");
151       }
152       if (!confirmacion) { // A6: el EB no confirma
153           pantalla.mostrarRegistroCancelado();
154           finCU();
155           return;
156       }
157       try {
158           actualizarBolsin();                                                          ◄ 42
159           llamarCasoUso29();                                                           ◄ 65
160           pantalla.mostrarEjecucionExitosa();                                          ◄ 66
161           // «include» 29. Notificar recepción de Bolsín: una notificación por cada CM de or…
162           notificacionesCU29.forEach(interfazNotificacion::notificarRecepcionBolsin);  ◄ 67
163       } catch (RuntimeException e) {
164           finCU(); // la transacción se revierte; se descartan los objetos que quedaron a me…
165           throw e;
166       }
167       finCU();                                                                         ◄ 68
168   }
```

`gestor/GestorRegistroRecepcionBolsines.java · actualizarBolsin`

```
239   private void actualizarBolsin() {
240       for (EstadoBolsin estado : estadoBolsinRepositorio.findAll()) {
241           if (estado.esRecibidoEnCMDestino()) {                                    ◄ 43
242               estadoRecibidoEnCMDestino = estado;
243           }
244       }
245       for (EstadoRemito estado : estadoRemitoRepositorio.findAll()) {
246           if (estado.esRecibidoyAceptado()) {                                      ◄ 44
247               estadoRecibidoYAceptado = estado;
248           }
249       }
250       fechaHoraActual = getFechaHoraActual();                                      ◄ 45
251       if (nrosDocumentacionMarcada == null) {
252           bolsinSeleccionado.actualizarEstado(fechaHoraActual, estadoRecibidoEnCMDestino, es…  ◄ 46
253                   encargado);
254       } else {
255           actualizarBolsinConDocumentacionMarcada();
256       }
257       // Persistencia (no es un mensaje del diagrama). saveAndFlush escribe todo ahora, ante…  ◄ —
258       // llamarCasoUso29(): un error de base revierte la transacción sin haber notificado.
259       bolsinRepositorio.saveAndFlush(bolsinSeleccionado);
260   }
```

`modelo/bolsin/Bolsin.java · actualizarEstado`

```
122   public void actualizarEstado(LocalDateTime fhActual, EstadoBolsin estadoBolsin, EstadoRemi…
123           Empleado responsable) {
124       actualizarRemitos(fhActual, estadoRemito, responsable);                      ◄ 47
125       for (CambioEstadoBolsin ce : cambioEstado) {
126           if (ce.sosActual()) {
127               ce.setFechaHoraFin(fhActual);                                        ◄ 62
128           }
129       }
130       cambioEstado.add(crearCambioEstado(fhActual, estadoBolsin, responsable));    ◄ 63
131   }
```

#	Mensaje (emisor → receptor)	Se envía en	Implementado en	Nota
40	`tomarConfirmacionRegistro(confirmacion: boolean)` EB → Pantalla	api.js:24	**Pantalla**:124	
41	`tomarConfirmacionRegistro(confirmacion: boolean)` Pantalla → Gestor	**Pantalla**:125	**Gestor**:145	Con false es A6. Todo en una transacción.
42	`actualizarBolsin()` Gestor → Gestor	**Gestor**:158	**Gestor**:239	
43	`*esRecibidoEnCMDestino(): boolean` Gestor → :EstadoBolsin (loop)	**Gestor**:241	**EstadoBolsin**:28	
44	`*esRecibidoyAceptado(): boolean` Gestor → :EstadoRemito (loop)	**Gestor**:246	**EstadoRemito**:23	
45	`getFechaHoraActual(): DateTime` Gestor → Gestor	**Gestor**:250	**Gestor**:229	Sale del bean Clock.
46	`actualizarEstado(fhActual, estadoBolsin, estadoRemito, responsable)` Gestor → selec:Bolsin	**Gestor**:252	**Bolsin**:122	
47	`actualizarRemitos(fhActual, estadoRemito, responsable)` Bolsin → Bolsin	**Bolsin**:124	**Bolsin**:137	
48	`actualizarEstado(estadoRemito, fhActual, responsable)` Bolsin → :Remito (loop)	**Bolsin**:139	**Remito**:78	
49	`setEstado(e: EstadoRemito)` Remito → Remito	**Remito**:79	**Remito**:84	
50	`actualizarDocumentacion(fhActual, responsable)` Remito → Remito	**Remito**:80	**Remito**:92	
51	`actualizarDocumentacion(fhActual, responsable)` Remito → :DetalleRemito (loop)	**Remito**:94	**DetalleRemito**:48	
52	`recibirYAceptar(fhActual, responsable)` DetalleRemito → :Documentacion	**DetalleRemito**:49	**Documentacion**:84	Entra al State.
53	`recibirYAceptar(this, historial, fhActual, responsable)` Documentacion → EnBolsinEnviado	**Documentacion**:85	**EnBolsinEnviado**:28	Mensaje polimórfico.
54	`new(): RecibidaYAceptada` EnBolsinEnviado → RecibidaYAceptada	**EnBolsinEnviado**:30	**Recibida YAceptada**:11	
55	`cambiarEstado(this, historial, fhActual, responsable, nuevo)` EnBolsinEnviado → sí mismo	**EnBolsinEnviado**:30	**Estado Documentacion**:62	Heredado de la abstracta.
56	`*sosActual(): boolean` → :CambioEstadoDocumentacion	**Estado Documentacion**:65	**CambioEstado Documentacion**:58	
57	`setFechaHoraFin(fechaHoraFin: DateTime)` → Actual:CambioEstadoDocumentacion	**Estado Documentacion**:66	**CambioEstado Documentacion**:53	
58	`crearCambioEstado(fhInicio, nuevo, responsable)` → sí mismo	**Estado Documentacion**:69	**Estado Documentacion**:79	
59	`new(fhInicio, e, responsable)` → Nuevo:CambioEstadoDocumentacion	**Estado Documentacion**:81	**CambioEstado Documentacion**:46	
60	`setEstado(e: EstadoDocumentacion)` → :Documentacion	**Estado Documentacion**:70	**Documentacion**:110	
61	`agregarCE(ce: CambioEstadoDocumentacion)` → :Documentacion	**Estado Documentacion**:71	**Documentacion**:115	
62	`setFechaHoraFin(fhFin: DateTime)` Bolsin → Actual:CambioEstadoBolsin	**Bolsin**:127	**Cambio EstadoBolsin**:46	Lo encuentra con sosActual() (sección 7).

#	Mensaje (emisor → receptor)	Se envía en	Implementado en	Nota
63	`crearCambioEstado(fhInicio, e, responsable): CambioEstadoBolsin` Bolsin → Bolsin	**Bolsin**:130	**Bolsin**:148	
64	`new(fhInicio, e, usuarioResponsable)` Bolsin → Nuevo:CambioEstadoBolsin	**Bolsin**:149	**Cambio EstadoBolsin**:39	
—	`saveAndFlush(bolsinSeleccionado)` Gestor → repositorio	**Gestor**:257	Spring Data	Persistencia: no está en el diagrama.
65	`llamarCasoUso29()` Gestor → Gestor	**Gestor**:159	**Gestor**:267	Prepara una notificación por CM de origen.
66	`mostrarEjecucionExitosa()` Gestor → Pantalla	**Gestor**:160	**Pantalla**:130	
67	`«include» 29. Notificar recepción de Bolsín` Gestor → CU 29	**Gestor**:162	**Notificacion BolsinLog**:20	Si falla, se revierte todo.
68	`FinCU()` Gestor → Gestor	**Gestor**:167	**Gestor**:277	

6. Flujos alternativos

Son una demostración del patrón: el diagrama modela el flujo descripto. Se agregaron sin cambiar ningún mensaje dibujado y sin clases nuevas, con métodos al final de cada archivo. El EB elige la opción, marca la documentación y confirma; a la marcada se le aplica otra transición del State y la no marcada se recibe como en el flujo descripto.

Flujo	Qué pasa	Transición (EnBolsinEnviado)	Remito
A1 / A2	El filtro por precinto o CM origen no encuentra nada (lo resuelve el front).	—	—
A3 · opción 2	La documentación marcada no llegó.	registrarNoRecibida → NoRecibida (**EnBolsinEnviado**:48)	Aceptado parcial
A4 · opción 3	La marcada no corresponde a la CM destino.	recibirYRechazar → RecibidaYRechazada (**EnBolsinEnviado**:38)	Aceptado parcial
A5 · opción 4	Sólo si el bolsín llega a la Central; la marcada tiene que tener área de la CMC.	redirigir → ParaRedirigir (**EnBolsinEnviado**:59)	Aceptado
A6	El EB no confirma: no se modifica nada.	**Gestor**:152	—

Recorrido en el código: **Gestor**:130 va a los alternativos → **Gestor**:309 (**switch**: qué pedirle al EB) → el EB marca → **Gestor**:332 valida con `admiteSeleccion` → al confirmar, **Gestor**:352 → **Bolsin**:194 → **Remito**:139 → **DetalleRemito**:80 (**switch**: qué transición). El switch es sobre la opción que eligió el EB, no sobre el estado. A5 se muestra con el perfil cmc: `.\mvnw.cmd spring-boot:run "-Dspring-boot.run.profiles=cmc"`.

7. Lo que no es idéntico al diagrama

El código sigue el diagrama de secuencia mensaje por mensaje. Hay dos lugares donde un mensaje dibujado no se puede escribir tal cual en el código; se implementan de una forma equivalente.

1. getNombre() dentro de obtenerCM()

En el diagrama	En el código

`Gestor → Logueado:Empleado: obtenerCM(): ComisionMedica`
`Empleado → :ComisionMedica: getNombre(): String`
`Gestor → Pantalla: mostrarCM(nombre: String)`
	`Gestor:195 cmUsuario = encargado.obtenerCM();`
	`Gestor:90 pantalla.mostrarCM(cmUsuario.getNombre());`
	`Empleado:43 obtenerCM() devuelve la CM`

Por qué: el Gestor necesita dos cosas de la CM del usuario: el **objeto**, para preguntarle a cada bolsín `esCmDestino(cm: ComisionMedica)`, y el **nombre**, para `mostrarCM(nombre)`. Un método devuelve una sola cosa: si obtenerCM() devolviera el nombre, el Gestor se quedaría sin el objeto y esCmDestino no cumpliría su firma. Por eso obtenerCM() devuelve la CM, como dice el diagrama, y el getNombre() lo envía el Gestor.

> **Cómo decirlo en la defensa:** "obtenerCM() devuelve la comisión médica, como dice su firma. El Gestor la necesita para esCmDestino y le pide el nombre para mostrarlo. Ya conocía a ComisionMedica, así que no se agrega acoplamiento."

2. Cómo encuentra el bolsín a "Actual:CambioEstadoBolsin"

En el diagrama	En el código

`selec:Bolsin → Actual:CambioEstadoBolsin: setFechaHoraFin(fhFin: DateTime)`
`(el diagrama ya sabe cuál es el actual)`
	`Bolsin:125 for (ce : cambioEstado)`
	`Bolsin:126 if (ce.sosActual())`
	`Bolsin:127 ce.setFechaHoraFin(fhActual);`

Por qué: el bolsín tiene una lista de cambios de estado. Para mandarle setFechaHoraFin al "Actual" primero tiene que saber cuál es: se lo pregunta a cada uno con sosActual(). Es la misma búsqueda que el diagrama dibuja en buscarBolsinesEnviados() (*sosActual()) y en la parte de Documentación, antes de su setFechaHoraFin.

> **Cómo decirlo en la defensa:** "El nombre Actual: de la línea de vida es el cambio de estado que no tiene fecha de fin. El bolsín lo encuentra preguntando sosActual(), igual que en buscarBolsinesEnviados()."

8. ¿Dónde está…? Índice por clase

Todos los métodos del back, clase por clase, con la línea donde empiezan. La columna "Origen" dice si el método es un mensaje del diagrama de secuencia, una operación del diagrama de clases, o un agregado.

Pantalla (PantallaRegistroRecepcionBolsines)

`pantalla/PantallaRegistroRecepcionBolsines.java`

Método	Línea	Vis.	Origen
`opcRegistrarRecepcionBolsin()`	52	+	Secuencia
`habilitarPantalla()`	59	+	Secuencia
`mostrarCM(String nombre)`	69	+	Secuencia
`mostrarBolsinesEnviadosParaSeleccion(int[] nroPrecinto, String[] origen)`	78	+	Secuencia
`tomarSeleccionBolsin(@RequestBody SeleccionBolsinRequest request)`	86	+	Secuencia
`mostrarNumRemitosyDocumentacionAsociada(int[] nrosRemito, String[][] documentacion)`	96	+	Secuencia
`mostrarOpcionesRecepcionParaSeleccion(String[] opciones)`	101	+	Secuencia
`tomarSeleccionOpcionRecepcion(@Valid @RequestBody SeleccionOpcionRequest request)`	107	+	Secuencia
`solicitarConfirmacionRegistro()`	114	+	Secuencia
`tomarConfirmacionRegistro(@RequestBody ConfirmacionRequest request)`	124	+	Secuencia
`mostrarEjecucionExitosa()`	130	+	Secuencia

Método	Línea	Vis.	Origen
`solicitarSeleccionDocumentacion(String mensajeSeleccion, String[][] documentacion)`	142	+	Alternativos A3-A5
`tomarSeleccionDocumentacion(@Valid @RequestBody SeleccionDocumentacionRequest request)`	152	+	Alternativos A3-A5
`mostrarRegistroCancelado()`	158	+	Alternativo A6

Gestor (GestorRegistroRecepcionBolsines)

`gestor/GestorRegistroRecepcionBolsines.java`

Método	Línea	Vis.	Origen
`opcRegistrarRecepcionBolsin()`	86	+	Secuencia
`tomarSeleccionBolsin(int nroPrecinto)`	100	+	Secuencia
`tomarSeleccionOpcionRecepcion(String opcion)`	121	+	Secuencia
`tomarConfirmacionRegistro(boolean confirmacion)`	145	+	Secuencia
`buscarUsuarioLogueado()`	175	–	Secuencia
`buscarCMUsuario()`	194	–	Secuencia
`buscarBolsinesEnviados()`	202	–	Secuencia
`buscarRemitos()`	214	–	Secuencia
`buscarDocumentacion()`	219	–	Secuencia
`getOpcionesRecepcion()`	224	–	Secuencia
`getFechaHoraActual()`	229	–	Secuencia
`actualizarBolsin()`	239	–	Secuencia
`llamarCasoUso29()`	267	–	Secuencia
`finCU()`	277	–	Secuencia
`olvidarOpcion()`	289	–	Apoyo
`getNumeroOpcion(String opcion)`	301	–	Alternativos A3-A5
`solicitarSeleccionDocumentacion(String opcion)`	309	–	Alternativos A3-A5
`tomarSeleccionDocumentacion(int[] nrosDocumentacion)`	332	+	Alternativos A3-A5
`actualizarBolsinConDocumentacionMarcada()`	352	–	Alternativos A3-A5

Sesion

`modelo/usuario/Sesion.java`

Método	Línea	Vis.	Origen
`getUsuario()`	31	+	Secuencia

Usuario

`modelo/usuario/Usuario.java`

Método	Línea	Vis.	Origen
`getUsuario()`	25	+	Secuencia

Empleado

`modelo/usuario/Empleado.java`

Método	Línea	Vis.	Origen
`esTuUsuario(Usuario usuario)`	35	+	Secuencia
`obtenerCM()`	43	+	Secuencia

ComisionMedica

`modelo/comisionmedica/ComisionMedica.java`

Método	Línea	Vis.	Origen
`getNombre()`	32	+	Secuencia
`getEmail()`	37	+	Extensión CU 29
`esCentral()`	49	+	Alternativos A3-A5

Bolsin

`modelo/bolsin/Bolsin.java`

Método	Línea	Vis.	Origen
buscarBolsinesEnviados()	69	+	Secuencia
esCmDestino(ComisionMedica cm)	79	+	Secuencia
getNumPrecinto()	84	+	Secuencia
getOrigen()	92	+	Secuencia
buscarRemitos()	101	+	Secuencia
buscarDocumentacion()	109	+	Secuencia
actualizarEstado(LocalDateTime fhActual, EstadoBolsin estadoBolsin, EstadoRemito estadoRemito, Empleado responsable)	122	+	Secuencia
actualizarRemitos(LocalDateTime fhActual, EstadoRemito estadoRemito, Empleado responsable)	137	–	Secuencia
crearCambioEstado(LocalDateTime fhInicio, EstadoBolsin e, Empleado responsable)	148	–	Secuencia
buscarOrigenesDocumentacion()	155	+	Extensión CU 29
buscarSituacionDocumentacion(ComisionMedica cmOrigen)	160	+	Extensión CU 29
esDestinoCentral()	173	+	Alternativos A3-A5
buscarDocumentacionParaMarcar()	178	+	Alternativos A3-A5
admiteSeleccion(int nroDocumentacion, int opcionRecepcion)	185	+	Alternativos A3-A5
actualizarEstado(LocalDateTime fhActual, EstadoBolsin estadoBolsin, EstadoRemito estadoRemito, EstadoRemito estadoRemitoParcial, Empleado responsable, int opcionRecepcion, int[] nrosMarcados)	194	+	Alternativos A3-A5

CambioEstadoBolsin

modelo/bolsin/CambioEstadoBolsin.java

Método	Línea	Vis.	Origen
CambioEstadoBolsin(LocalDateTime fhInicio, EstadoBolsin e, Empleado usuarioResponsable)	39	+	Secuencia
setFechaHoraFin(LocalDateTime fechaHoraFin)	46	+	Secuencia
sosActual()	51	+	Secuencia
sosEnviado()	56	+	Secuencia

EstadoBolsin

modelo/bolsin/EstadoBolsin.java

Método	Línea	Vis.	Origen
sosEnviado()	23	+	Secuencia
esRecibidoEnCMDestino()	28	+	Secuencia

Remito

modelo/remito/Remito.java

Método	Línea	Vis.	Origen
getNumero()	56	+	Secuencia
buscarDatosDocumentacion()	65	+	Secuencia
actualizarEstado(EstadoRemito estadoRemito, LocalDateTime fhActual, Empleado responsable)	78	+	Secuencia
setEstado(EstadoRemito e)	84	+	Secuencia
actualizarDocumentacion(LocalDateTime fhActual, Empleado responsable)	92	–	Secuencia
getOrigen()	101	+	Extensión CU 29
buscarSituacionDocumentacion()	106	+	Extensión CU 29
buscarDocumentacionParaMarcar()	121	+	Alternativos A3-A5
admiteSeleccion(int nroDocumentacion, int opcionRecepcion)	131	+	Alternativos A3-A5
actualizarEstado(EstadoRemito estadoRemito, EstadoRemito estadoRemitoParcial, LocalDateTime fhActual, Empleado responsable, int opcionRecepcion, int[] nrosMarcados)	139	+	Alternativos A3-A5

DetalleRemito

modelo/remito/DetalleRemito.java

Método	Línea	Vis.	Origen
buscarDatosDocumentacion()	40	+	Secuencia
actualizarDocumentacion(LocalDateTime fhActual, Empleado responsable)	48	+	Secuencia
buscarSituacionDocumentacion()	55	+	Extensión CU 29
buscarDocumentacionParaMarcar()	66	+	Alternativos A3-A5
admiteSeleccion(int nroDocumentacion, int opcionRecepcion)	72	+	Alternativos A3-A5
actualizarDocumentacion(LocalDateTime fhActual, Empleado responsable, int opcionRecepcion, int[] nrosMarcados)	80	+	Alternativos A3-A5
dejaRemitoParcial(int opcionRecepcion, int[] nrosMarcados)	95	+	Alternativos A3-A5
estaMarcada(int[] nrosMarcados)	99	–	Alternativos A3-A5

EstadoRemito

modelo/remito/EstadoRemito.java

Método	Línea	Vis.	Origen
esRecibidoyAceptado()	23	+	Secuencia
esRecibidoYAceptadoParcial()	32	+	Alternativos A3-A5

Documentacion

modelo/documentacion/Documentacion.java

Método	Línea	Vis.	Origen
Documentacion(int numero, String asunto, LocalDate fechaPase, TipoDocumento tipoDocumento)	58	+	Constructor
getAsunto()	66	+	Secuencia
getNombreTipo()	71	+	Secuencia
recibirYAceptar(LocalDateTime fhActual, Empleado responsable)	84	+	Secuencia
recibirYRechazar(LocalDateTime fhActual, Empleado responsable)	92	+	Clases (State)
registrarNoRecibida(LocalDateTime fhActual, Empleado responsable)	97	+	Clases (State)
redirigir(LocalDateTime fhActual, Empleado responsable)	105	+	Clases (State)
setEstado(EstadoDocumentacion e)	110	+	Secuencia
agregarCE(CambioEstadoDocumentacion ce)	115	+	Secuencia
getNombreEstado()	125	+	Extensión CU 29
getNumero()	137	+	Alternativos A3-A5

CambioEstadoDocumentacion

modelo/documentacion/CambioEstadoDocumentacion.java

Método	Línea	Vis.	Origen
CambioEstadoDocumentacion(LocalDateTime fhInicio, EstadoDocumentacion e, Empleado responsable)	46	+	Secuencia
setFechaHoraFin(LocalDateTime fechaHoraFin)	53	+	Secuencia
sosActual()	58	+	Secuencia

TipoDocumento

modelo/documentacion/TipoDocumento.java

Método	Línea	Vis.	Origen
getNombre()	19	+	Secuencia

EstadoDocumentacion

modelo/documentacion/EstadoDocumentacion.java

Método	Línea	Vis.	Origen
EstadoDocumentacion(String nombre)	21	#	Código común
getNombre()	26	+	Clases (State)
recibirYAceptar(Documentacion doc, List<CambioEstadoDocumentacion> historial, LocalDateTime fhActual, Empleado responsable)	31	+	Clases (State)
recibirYRechazar(Documentacion doc, List<CambioEstadoDocumentacion> historial, LocalDateTime fhActual, Empleado responsable)	37	+	Clases (State)
registrarNoRecibida(Documentacion doc, List<CambioEstadoDocumentacion> historial, LocalDateTime fhActual, Empleado responsable)	43	+	Clases (State)
redirigir(Documentacion doc, List<CambioEstadoDocumentacion> historial, LocalDateTime fhActual, Empleado responsable)	49	+	Clases (State)
cambiarEstado(Documentacion doc, List<CambioEstadoDocumentacion> historial, LocalDateTime fhActual, Empleado responsable, EstadoDocumentacion nuevo)	62	#	Secuencia
crearCambioEstado(LocalDateTime fhInicio, EstadoDocumentacion nuevo, Empleado responsable)	79	#	Secuencia
transicionInvalida(String accion)	84	–	Código común

EnBolsinEnviado

modelo/documentacion/EnBolsinEnviado.java

Método	Línea	Vis.	Origen
EnBolsinEnviado()	18	+	Clases: new()
recibirYAceptar(Documentacion doc, List<CambioEstadoDocumentacion> historial, LocalDateTime fhActual, Empleado responsable)	28	+	Secuencia (State)
recibirYRechazar(Documentacion doc, List<CambioEstadoDocumentacion> historial, LocalDateTime fhActual, Empleado responsable)	38	+	Clases (State)
registrarNoRecibida(Documentacion doc, List<CambioEstadoDocumentacion> historial, LocalDateTime fhActual, Empleado responsable)	48	+	Clases (State)
redirigir(Documentacion doc, List<CambioEstadoDocumentacion> historial, LocalDateTime fhActual, Empleado responsable)	59	+	Clases (State)

ParaRedirigir

modelo/documentacion/ParaRedirigir.java

Método	Línea	Vis.	Origen
ParaRedirigir()	17	+	Clases: new()
recibirYAceptar(Documentacion doc, List<CambioEstadoDocumentacion> historial, LocalDateTime fhActual, Empleado responsable)	26	+	Clases (State)
recibirYRechazar(Documentacion doc, List<CambioEstadoDocumentacion> historial, LocalDateTime fhActual, Empleado responsable)	36	+	Clases (State)

RecibidaYRechazada

modelo/documentacion/RecibidaYRechazada.java

Método	Línea	Vis.	Origen
RecibidaYRechazada()	18	+	Clases: new()
redirigir(Documentacion doc, List<CambioEstadoDocumentacion> historial, LocalDateTime fhActual, Empleado responsable)	24	+	Clases (State)

EstadoDocumentacionConverter

modelo/documentacion/EstadoDocumentacionConverter.java

Método	Línea	Vis.	Origen
convertToDatabaseColumn(EstadoDocumentacion estado)	29	+	—
convertToEntityAttribute(String nombre)	34	+	—

NotificacionBolsinLog

`notificacion/NotificacionBolsinLog.java`

Método	Línea	Vis.	Origen
`notificarRecepcionBolsin(NotificacionRecepcionBolsin notificacion)`	20	+	Extensión CU 29

NotificacionRecepcionBolsin

`notificacion/NotificacionRecepcionBolsin.java`

Método	Línea	Vis.	Origen
`de(String emailCMOrigen, String nombreCMOrigen, int nroPrecinto, String nombreCMDestino, String[][] situacion)`	19	+	Extensión CU 29

ManejadorErrores

`pantalla/ManejadorErrores.java`

Método	Línea	Vis.	Origen
`datosInvalidos(Exception e)`	20	+	—
`estadoInvalido(IllegalStateException e)`	25	+	—

RelojConfig

`config/RelojConfig.java`

Método	Línea	Vis.	Origen
`reloj()`	16	+	—

Los estados Registrada, EnRemito, EnBolsinSaliente, RecibidaYAceptada, NoRecibida y DeBaja sólo tienen su constructor (el new() del diagrama de clases). Los repositorios (repositorio/) y los records (dto/) no tienen lógica del CU.

9. Persistencia y datos de prueba

Tema	Cómo está resuelto
Base y mapeo	H2 en modo archivo (back/data). JPA / Hibernate: cada entidad es una @Entity; consola en http://localhost:8080/h2-console (JDBC URL jdbc:h2:file:./data/bolsines, usuario sa, sin contraseña).
Estado de la documentación	Se guarda por nombre; EstadoDocumentacionConverter crea el estado concreto al leer.
Transacción	La confirmación es @Transactional: si falla la base o la notificación, no queda nada guardado.
Datos	data.sql se carga en cada arranque: al reiniciar el back, todo vuelve al estado inicial.
Pruebas	26 (State aislado, el CU completo contra H2, DTO y CU 29): .\mvnw.cmd clean install.

Precinto	Origen → destino	Para qué
5001, 5002, 5005	Central, Rosario y Mendoza → Córdoba	Demo con Laura Gómez (EB de Córdoba)
5007, 5008, 5009	Rosario, Central y Mendoza → Córdoba	Para probar más veces
5003 / 5004	Mendoza → Córdoba / Central → Rosario	No aparecen: cerrado / va a otra CM
5006 y 5010	Córdoba y Rosario → Central	Sólo con el perfil cmc (EB: Ana Ruiz). La 11 y la 112 no tienen área a propósito, para mostrar la validación de A5.

10. Preguntas típicas de la defensa

¿Dónde está el patrón State?
→ En modelo/documentacion. Documentacion es el contexto (**Documentacion**:84), EstadoDocumentacion el estado abstracto y hay 9 estados concretos; el del CU 28 es EnBolsinEnviado (**EnBolsinEnviado**:28).

¿Por qué State y no un if por estado?
→ Porque el comportamiento depende del estado. Con if, cada operación preguntaría el estado y agregar uno obligaría a modificarlas todas; con State cada estado implementa lo suyo.

¿Quién decide el estado siguiente?
→ El estado concreto: EnBolsinEnviado crea un RecibidaYAceptada (new) y llama a cambiarEstado.

¿Por qué herencia y no una interfaz?
→ Por el reuso de código: cambiarEstado y crearCambioEstado son iguales para todos los estados y están una sola vez, en la clase abstracta.

¿Qué pasa con una transición que no corresponde?
→ Responde la clase abstracta con un error (409) y no cambia nada.

¿Qué patrones GRASP se ven?
→ Controlador y Fabricación Pura (Gestor), Experto (cada entidad responde sobre sus datos), Creador (el Bolsín crea sus CambioEstadoBolsin), Bajo Acoplamiento y Alta Cohesión.

¿Por qué la Pantalla es un @RestController?
→ Es el «boundary»: recibe los mensajes del actor, con el mismo nombre del diagrama. React sólo dibuja.

¿Por qué A5 no funciona con Laura?
→ Porque A5 corresponde cuando el bolsín llega a la Central y Laura es EB de Córdoba. Es el mismo actor (EB), de otra CM: se muestra con el perfil cmc.

¿Por qué hay un switch si es orientado a objetos?
→ Es sobre la opción que elige el EB (la alternativa del CU), no sobre el estado; de los estados se encarga el State.

¿Dónde está la persistencia?
→ Entidades JPA, repositorios, H2 y data.sql. En /h2-console se ve cómo cambian los estados después de confirmar.